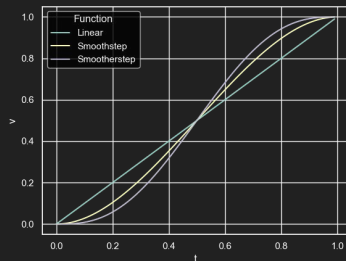


L06: Game UI

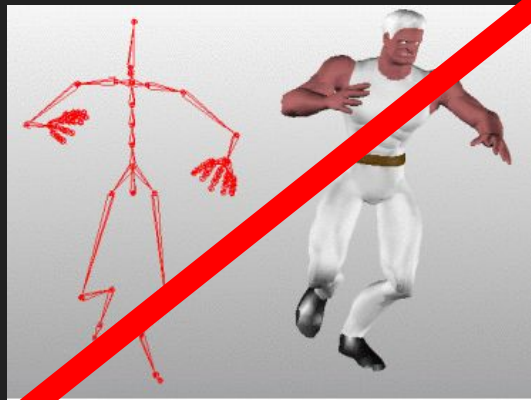
Chris Carvelli

Recap



```
float t = 0.5f;  
float v = smoothstep(t);  
vec2f pos = lerp(from, to, v);
```

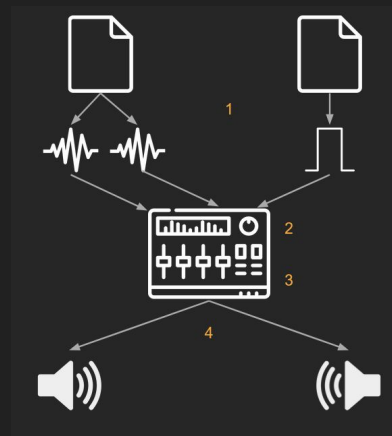
lerping and smoothing



~~3D animation~~
(postponed to L08)



2D animation



audio and SDL_mixer

Agenda

1. Graphics User Interfaces
2. Retained vs Immediate GUIs
3. imgui

Exercise 05 review

Debaucherous Demons
Critical ATK DEF AGI RPL

You're back in! Let's get even with them!



ORDER L2 Change Tactics
PERSONA Use a Skill

L06 - Game UI

Target Intel

SHOWTIME

593 328

519 374

422 294

SP 23 / 300

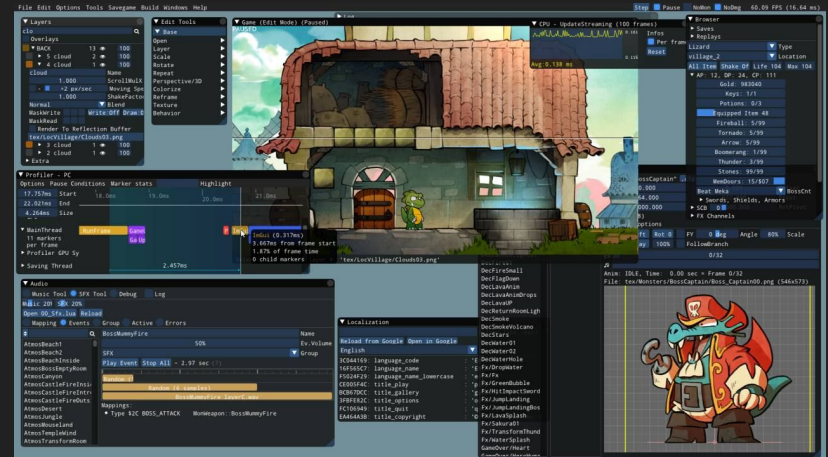
Rush Assist

Problem statement: How do we...



- ...create effective tools for game development?

- ...create dynamic and effective UIs for games?
- ...integrate our UIs with the game world?

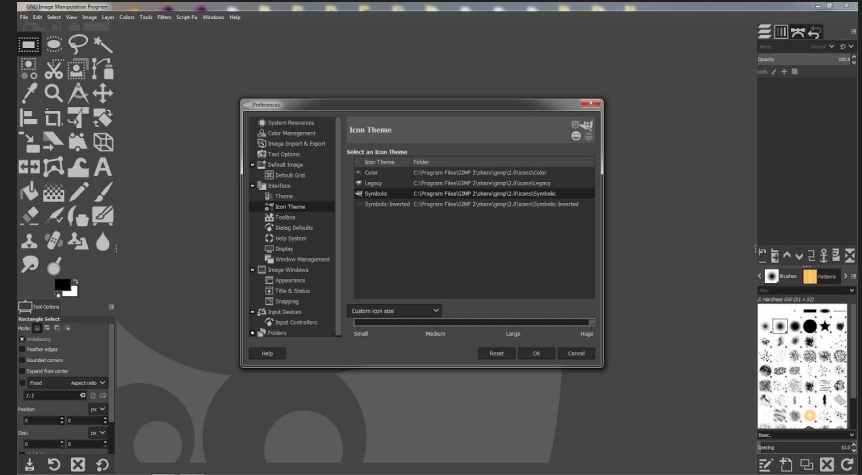


Agenda

1. Graphics User Interfaces
2. Retained vs Immediate GUIs
3. imgui

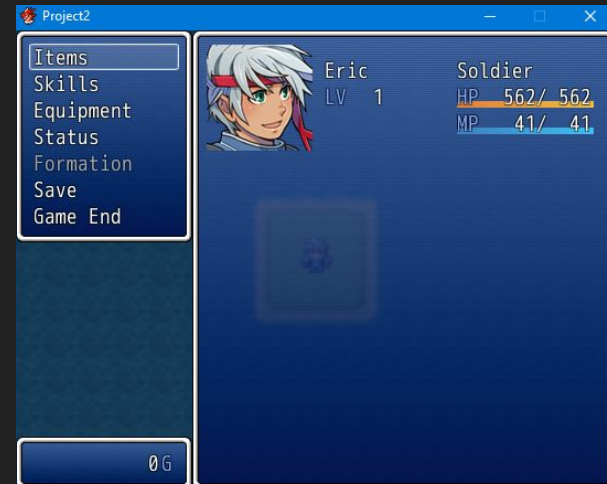
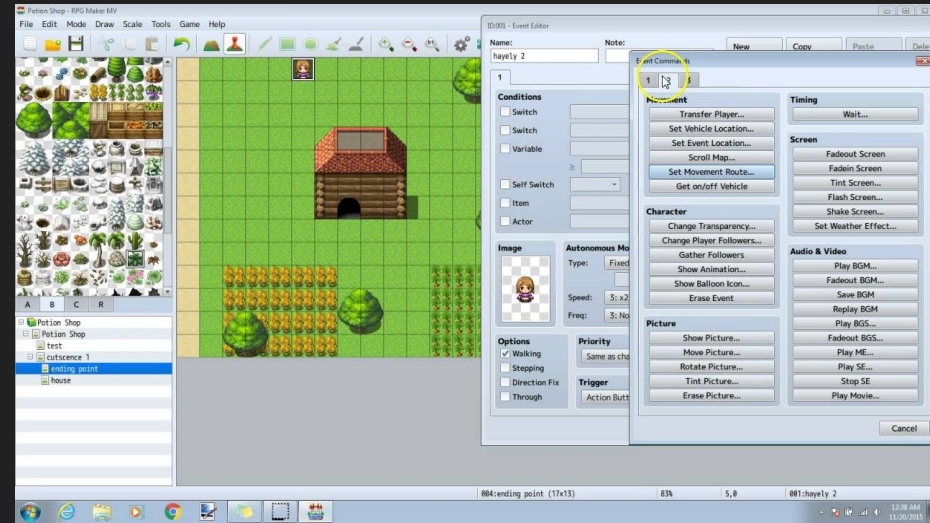
What is a GUI?

- Graphical User Interface is a broad concept that covers many aspects of **user interaction** and **visual feedback**.
- Commonly thought of as the **interactable part** of an application.
- Most **desktop applications** are pure **data-driven** GUI applications.
- **Games**, 3D modelling apps, drawing apps, etc. can be thought of as a **visualization with a GUI on top**.



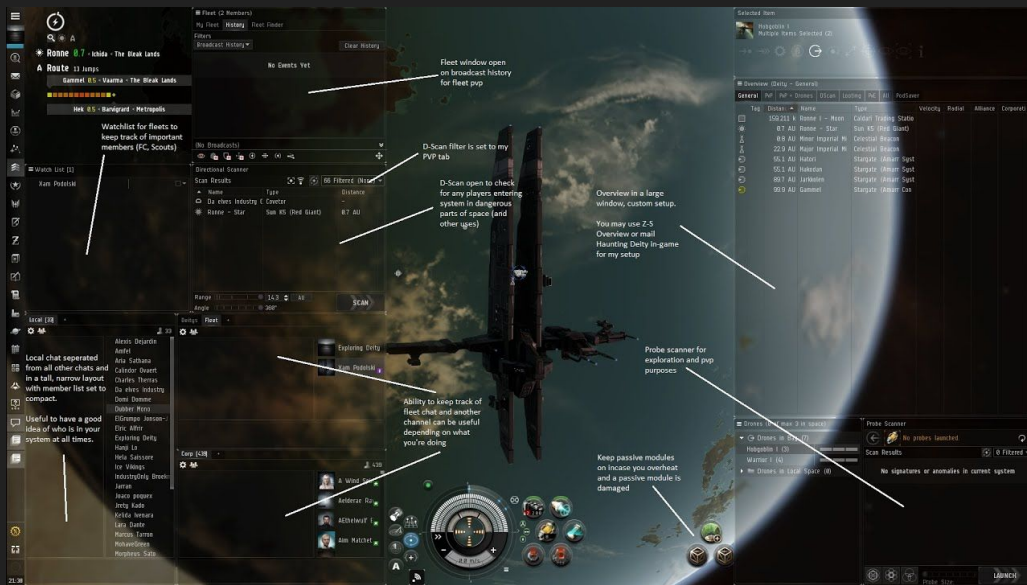
GUI in Game Engines

- In **Game Engines**, GUIs are mostly used as **tools** to create a game around them, or to debug stats and visuals.
- But **Games** themselves also have GUIs, to allow the user to interact with them.
- Engines have complex interactions but don't necessitate great graphics, games are usually the opposite.



Drawing GUI elements

- Typically GUI elements will be drawn last, allowing them to be drawn on top of everything.
- This means, draw all your 2D/3D stuff on screen, then draw HUD / GUI.
- Camera effects, can go either before the GUI or after, depending on whether you want the GUI to be affected by them.



Agenda

1. Graphics User Interfaces
2. Retained vs Immediate GUIs
3. imgui

What is RMGUI?

Retained Mode GUI (aka RMGUI).

- GUI elements **hold their own state**.
- Application sets GUI states.
- Application Receives callbacks from GUI.
- Object references or IDs are used to keep track of objects (aka **synchronisation of state**).

InputBox	
-	id : long
-	text : string
+	onInputChanged <event>
+	setValue(val : string)

GUI

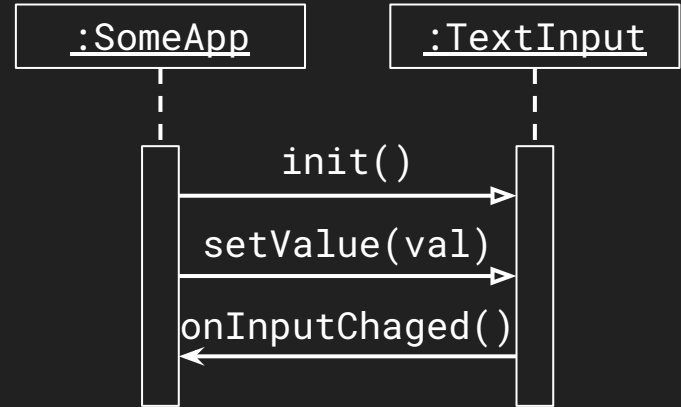
```
+ InputBox(val : &string)  
+ Button(lbl : string) : bool  
  [...]  
  
  // some API use return values  
  // instead of reference vals  
+ InputBox(val : string) : string
```

InputBox

```
- id    : long  
- text  : string  
  
+ onInputChanged <event>  
+ setValue(val : string)
```

Typical RMGUI usage

- `Init()`
 - Create Widgets & save IDs
 - Setup callback
- `Callback(int widgetID)`
 - Handle callback based on ID



What is ImGui?

Immediate Mode GUI (aka ImGui).

- GUI elements only exist as **function calls**.
- Application state is reflected directly in GUI.
- **No synchronisation** needed.
- Immediately respond to GUI input where it happens.

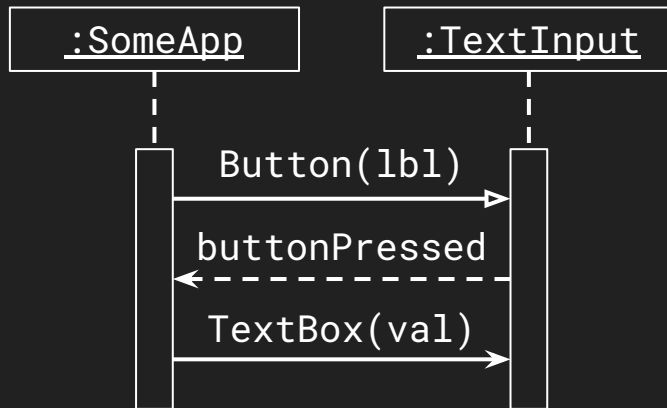
GUI

```
+ InputBox(val : &string)  
+ Button(lbl : string) : bool  
  [...]  
  
  // some API use return values  
  // instead of reference vals  
+ InputBox(val : string) : string
```


Typical ImGui usage

```
// in SomeApp::Update():
```

- `if(GUI::Button("Press Me"))
doSomething();`
- `GUI::TextBox(&editMe);`



RMGUI vs IMGUI

RMGUI

- Pros
 - Object oriented design
 - Declarative
 - Reusable components
- Cons
 - Lot of state duplication

IMGUI

- Pros
 - Procedural
 - No state duplication
 - More flexible
- Cons
 - Internally need to maintain state

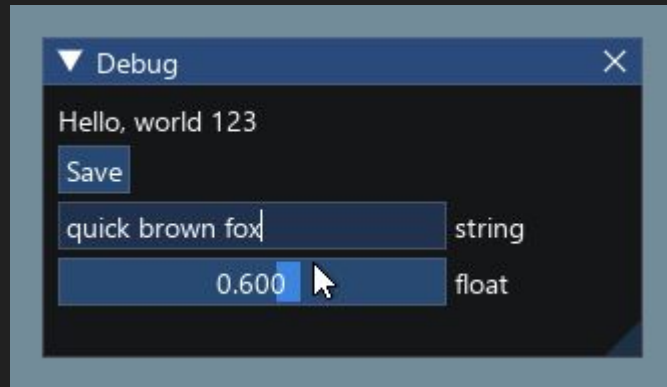
Agenda

1. Graphics User Interfaces
2. Retained vs Immediate GUIs
3. imgui

What is Dear ImGui?

“Dear ImGui is a bloat-free graphical user interface library for C++. It outputs optimized vertex buffers that you can render anytime in your 3D-pipeline-enabled application. It is fast, portable, renderer agnostic, and self-contained (no external dependencies).”

- Immediate mode GUI
- Values to ImGui are passed as pointers (not copies).
- ImGui will return true when a value has changed.



```
ImGui::Text("Hello world %d", 123);

if (ImGui::Button("Ok")) {
    // do stuff
}

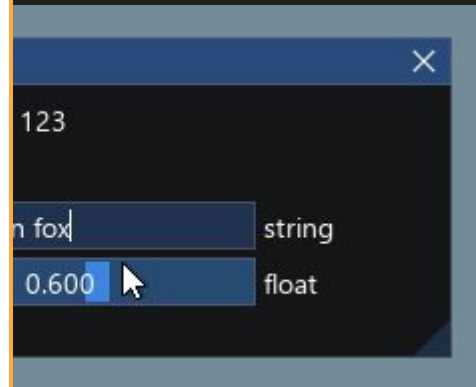
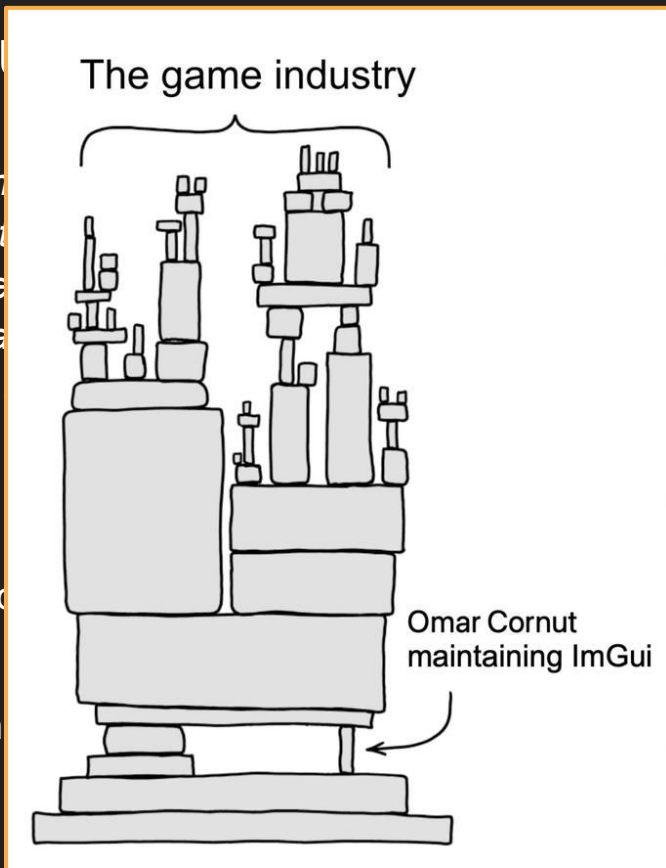
char buf[256] = "quick brown fox";
ImGui::InputText("string", buf, 256);

float f = 0.6f;
ImGui::SliderFloat("float", &f, 0.0f, 1.0f);
```

What is Dear ImGui

"Dear ImGui is a bloat-free graph interface library for C++. It outputs vertex buffers that you can render in your 3D-pipeline-enabled application. It is portable, renderer agnostic, and has no external dependencies."

- Immediate mode GUI
- Values to ImGui are passed by reference (not copies).
- ImGui will return true when the window is changed.

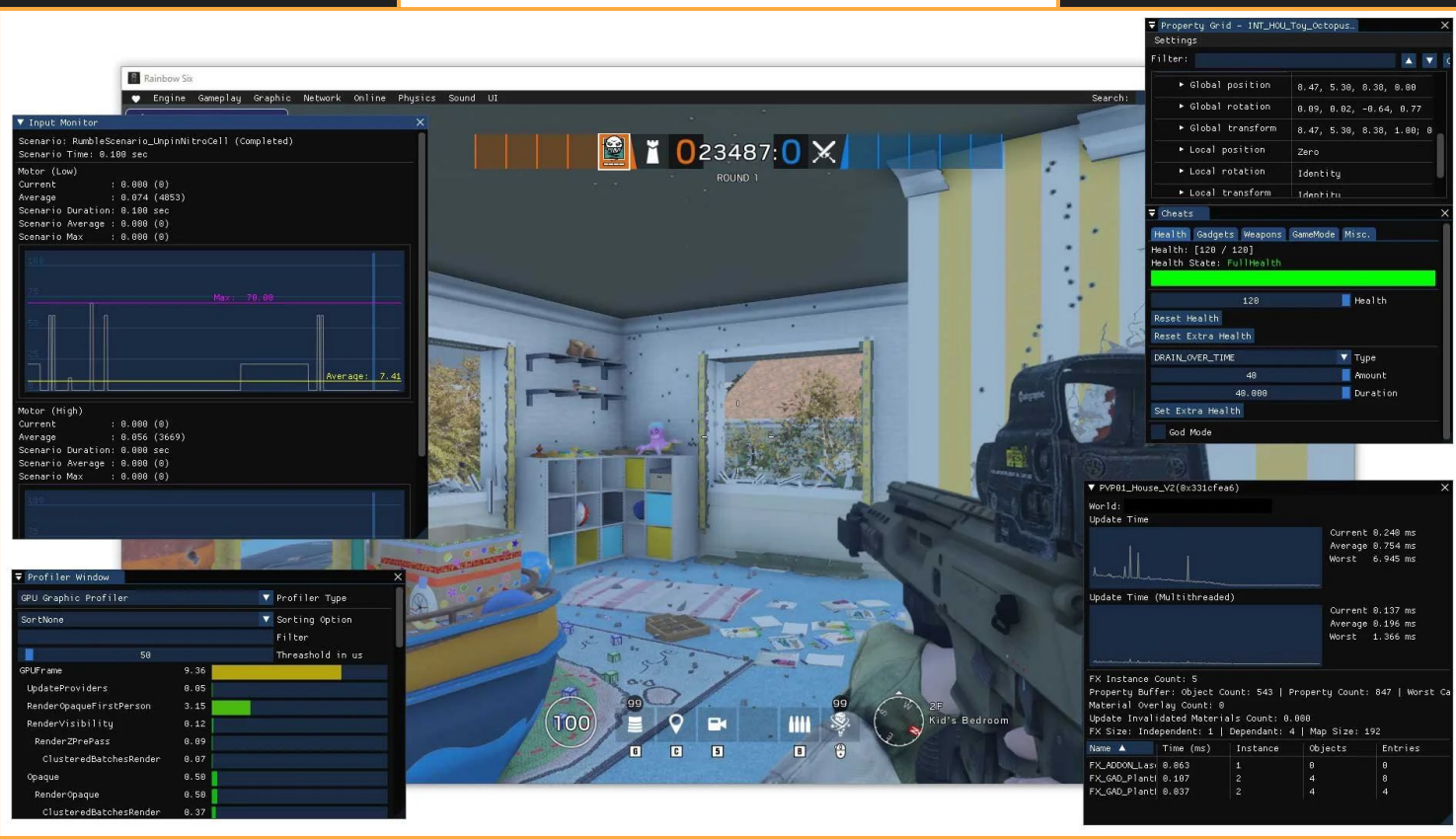


```
world %d", 123);  
Ok")) {  
    ImGuiTextBuffer buf;  
    ImGuiTextBuffer::append("pick brown fox");  
    ImGuiTextBuffer::append("string", buf, 256);  
    ImGuiTextBuffer::append("float", &f, 0.0f, 1.0f);
```

What

“Dear ImGui
interface
vertex b
your 3D
portable
(no exte

- Im
- Va
(no
- Im
ch



<https://github.com/ocornut/imgui>

What

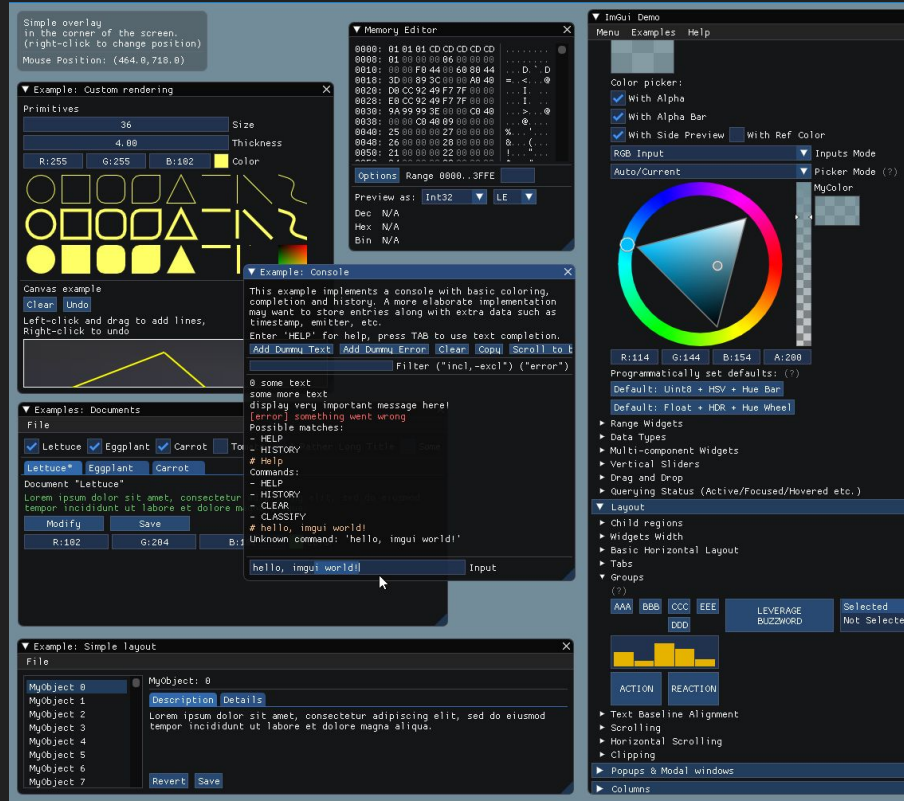
“Dear ImGui
interface
vertex b
your 3D
portable
(no exte

- Im
- Va
(no
- Im
ch



(gallery of cool stuff) <https://github.com/ocornut/imgui/issues/5243>
<https://github.com/ocornut/imgui/wiki/Useful-Extensions>

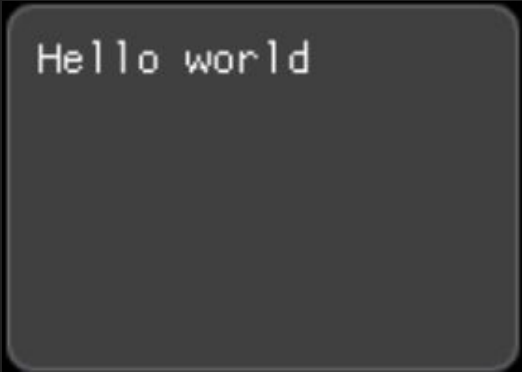
It does all sorts of cool stuff



Customizing windows

- ImGui Windows can be resized and collapsed.
- Their behaviour can be redefined using `ImGuiWindowFlags_`.

```
ImGuiWindowFlags flags = ImGuiWindowFlags_NoTitleBar |  
                          ImGuiWindowFlags_NoResize  |  
                          ImGuiWindowFlags_NoMove;  
  
bool open = false;  
ImGui::Begin("", &open, flags);  
ImGui::Text("Hello world");  
ImGui::End();
```



Hello world

Set the position of windows

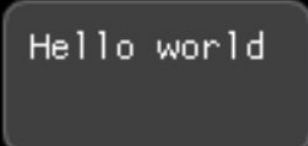
```
ImGuiCond cond = ImGuiCond_Always;

ImVec2 pos    = { 0, 0 };
ImVec2 pivot  = { 0, 0 };
ImVec2 size   = { 100, 50 };

// these will be called only if `cond` matches
// (in this case always)
ImGui::SetNextWindowPos(pos, cond, pivot);
ImGui::SetNextWindowSize(size, cond);

ImGuiWindowFlags flags = ImGuiWindowFlags_NoTitleBar |
                          ImGuiWindowFlags_NoResize  |
                          ImGuiWindowFlags_NoMove;

bool* open = NULL;
ImGui::Begin("New window", open, flags);
ImGui::Text("Hello world");
ImGui::End();
```

A screenshot of a window titled "New window" with the text "Hello world" inside. The window has a dark gray title bar and a light gray body. The text "Hello world" is displayed in a monospaced font.

Set the position of windows - conventions

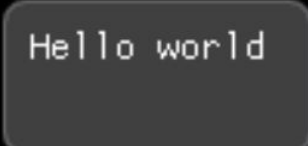
```
ImGuiCond cond = ImGuiCond_Always;

ImVec2 pos { WINDOW_W, WINDOW_H };
ImVec2 pivot = { 1, 1 }; // (0,0) upper left
                        // (1,1) lower right
ImVec2 size = { 100, 50 };

ImGui::SetNextWindowPos(pos, cond, pivot);
ImGui::SetNextWindowSize(size, cond);

ImGuiWindowFlags flags = ImGuiWindowFlags_NoTitleBar |
                          ImGuiWindowFlags_NoResize |
                          ImGuiWindowFlags_NoMove;

bool* open = NULL;
ImGui::Begin("New window", open, flags);
ImGui::Text("Hello world");
ImGui::End();
```



Hello world

Set the style of windows - conventions

```
// transparent color
ImVec4 col = ImVec4(0.0f, 0.0f, 0.0f, 0.0f);

// setup window as in previous slides

bool* open = nullptr;
ImGui::PushStyleColor(ImGuiCol_WindowBg, col);
ImGui::PushStyleColor(ImGuiCol_Border, col);

ImGui::Begin("", open, flags);
ImGui::Text("Hello world");
ImGui::End();

ImGui::PopStyleColor();
ImGui::PopStyleColor();

// NOTE: for transparency, a better way would be
// ImGuiWindowFlags flags =
//     ImGuiWindowFlags_NoBackground |
//     ImGuiWindowFlags_NoDecoration;
```

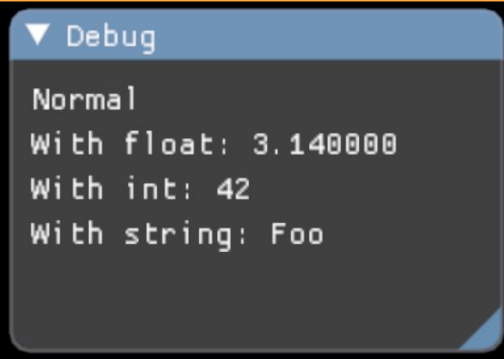
Hello world

Working with text

Text (and many other ImGui functions) work similar to the C-style printf function and can format text-strings:

- %f : float
- %i : int
- %s : string

```
ImGui::Text("Normal");  
  
float f1 = 3.14;  
ImGui::Text("With float: %f", f1);  
  
int in = 42;  
ImGui::Text("With int: %i", in);  
  
const char* st = "Foo";  
ImGui::Text("With string: %s", st);
```



<https://cplusplus.com/reference/cstdio/printf/>

Text alignment

- We can explicit set the cursor's position to specify where the next item will be rendered.
- With a bit of math, we can use this to do alignment of items.

```
ImGui::Text("Left aligned");

float width = ImGui::CalcTextSize("Centered").x;
float windowWidth =
ImGui::GetWindowContentRegionWidth();
ImGui::SetCursorPosX((windowWidth - width)/2);
ImGui::Text("Centered");

width = ImGui::CalcTextSize("Right aligned").x;
ImGui::SetCursorPosX(windowWidth - width);
ImGui::Text("Right aligned");
```



Image button

```
ImVec2 size = ImVec2 { 200,75 };  
SDL_Texture* tex_play;  
SDL_Texture* tex_sett;  
  
if (ImGui::ImageButton("btn_play", tex_play, size)  
    playPressed());  
  
if (ImGui::ImageButton("btn_sett", tex_sett, size)  
    settingsPressed());
```



Custom fonts

```
// Setup custom TrueType Font
auto fonts = ImGui::GetIO().Fonts;
fonts->AddFontDefault();
auto fontName = "assets/ProggyTiny.ttf";
float fontSize = 20;
ImFont* ProggyTiny =
fonts->AddFontFromFileTTF(fontName, fontSize);

// Use custom font
ImGui::PushFont(ProggyTiny);
ImGui::Text("Hello world");
ImGui::PopFont();
```



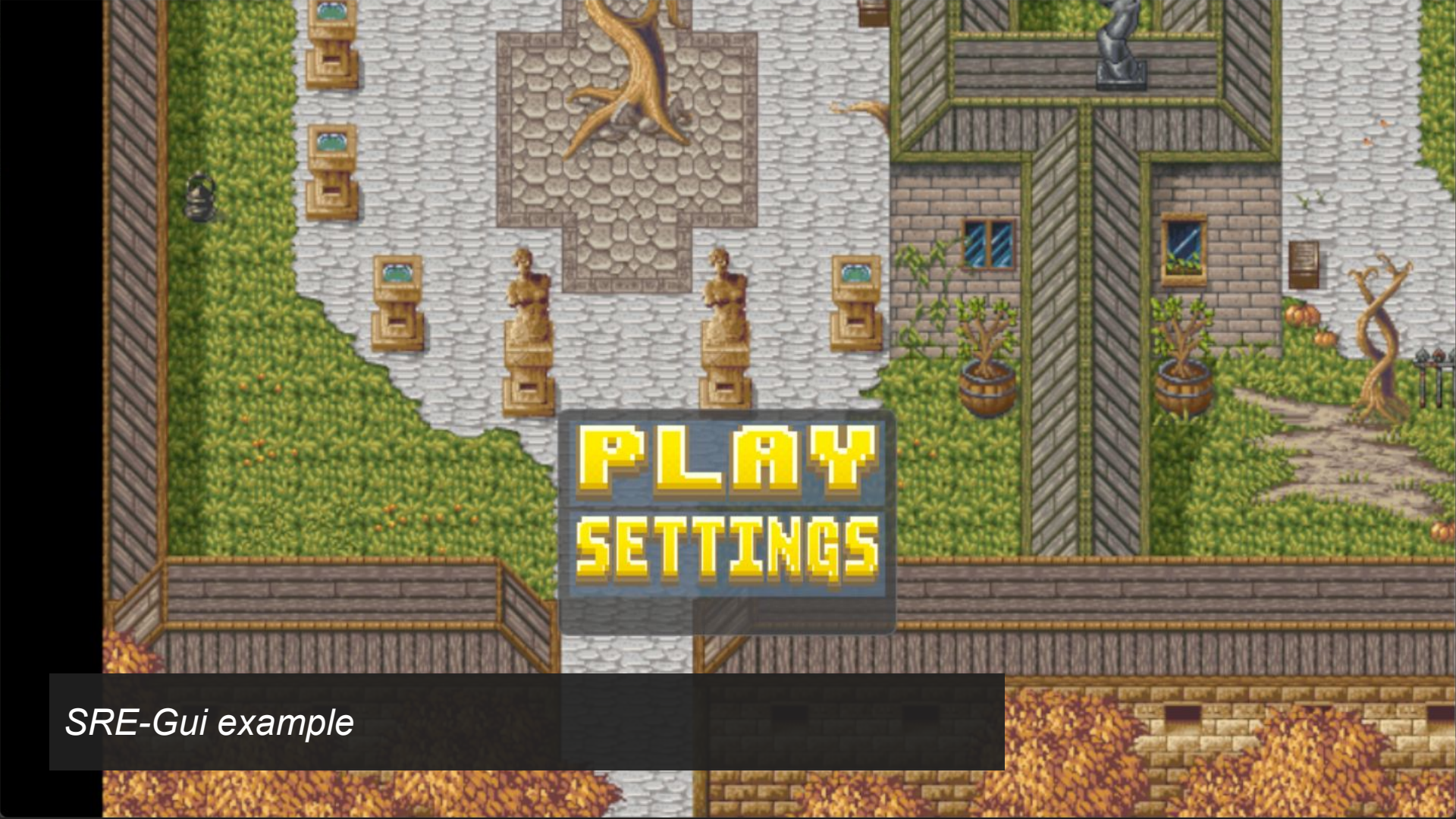
Working with explicit ids

- Internally ImGui uses the label-string to store the state of an object (e.g. is selected)
- This can cause conflicts if same label is used multiple times
- Fix
 - Append “##some-id” to label
 - Use PushId / PopId

```
if (ImGui::Button("Meow"))
    std::cout << "Meow\n"; // prints "Meow"
if (ImGui::Button("Meow"))
    std::cout << "Purr\n"; // does not work!

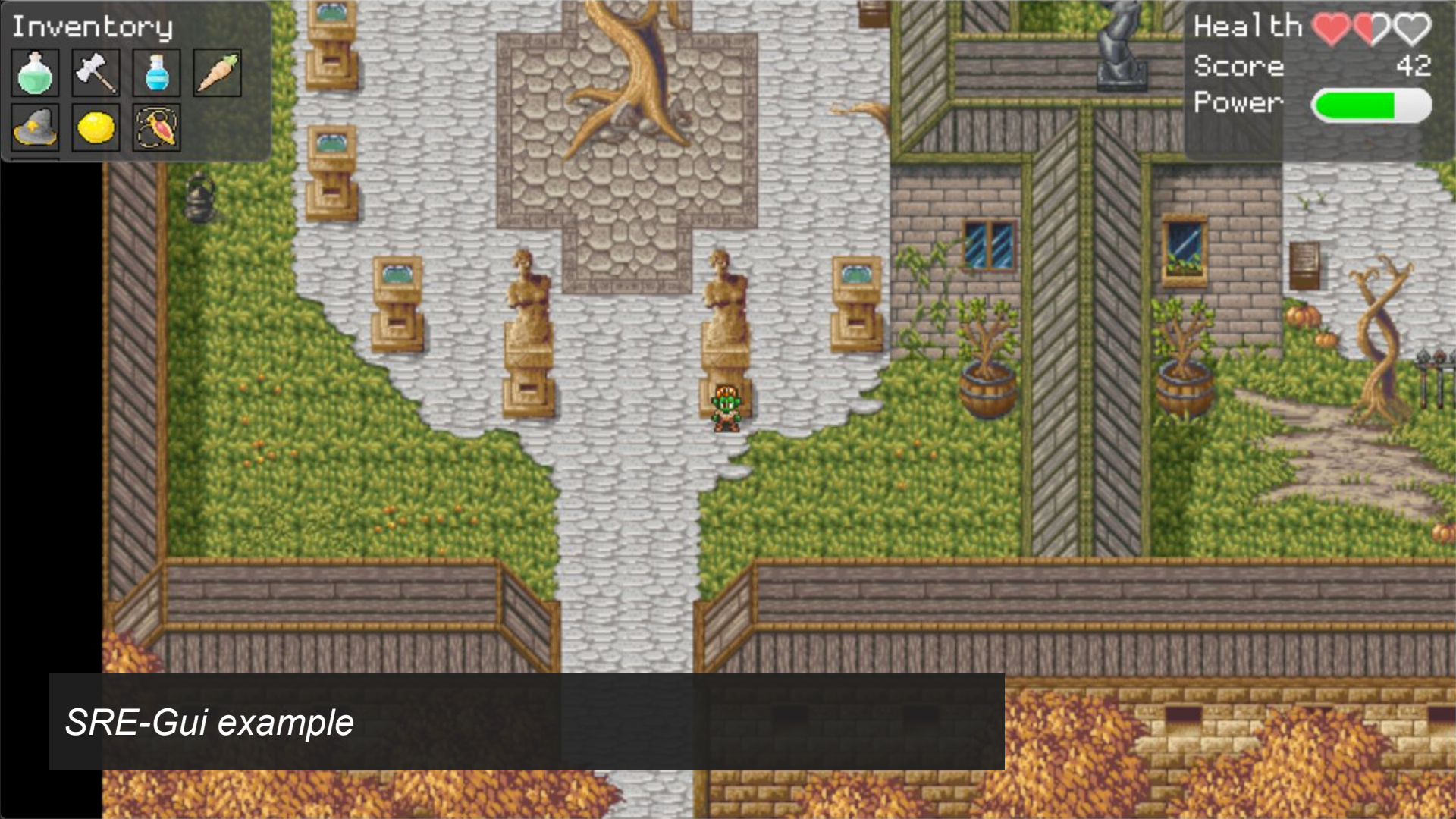
// append id to label
if (ImGui::Button("Meow"))
    std::cout << "Meow\n";
if (ImGui::Button("Meow##Second"))
    std::cout << "Purr\n";

// push/pop id
for (int i = 0; i < 3; ++i) {
    ImGui::PushID(i);
    ImGui::InputInt("##", &arr[i]);
    ImGui::PopID();
}
```

A pixel art game scene featuring a menu overlay. The background shows a stone-paved courtyard with a large stone cross in the center. To the left is a grassy area with a small black cat. To the right is a stone building with windows and potted plants. In the foreground, there are autumn leaves. The menu overlay is a dark grey rectangle with the words 'PLAY' and 'SETTINGS' in large, yellow, pixelated font.

PLAY
SETTINGS

SRE-Gui example



SRE-Gui example